# Email Service

CSYE 6225 - Boston

Spring 2026

Tejas Parikh (t.parikh@northeastern.edu)

# Learning Objectives

- Explain why cloud applications use managed email services instead of self-hosted mail servers
- Describe the email delivery lifecycle from sender to recipient inbox
- Configure and explain SPF, DKIM, and DMARC authentication mechanisms
- Identify how email bounces, complaints, and spam traps damage sender reputation
- Compare AWS SES with GCP's email sending options and third-party services
- Analyze real-world email misconfiguration incidents and their business impact
- Design a cloud-native email sending architecture with proper authentication and reputation monitoring

# Why Cloud Applications Need Email Services

# The Role of Email in Cloud Applications

- Email is not just marketing newsletters. In a cloud-native application, email serves three critical functions:
  - **Transactional messages** — Purchase confirmations, shipping notifications, order status updates, password resets, two-factor authentication codes, and policy change notices. These are triggered by user actions and are time-sensitive.
  - **Notification messages** — System health reports, application alerts, workflow status updates, billing reminders, and account activity summaries. These are triggered by system events.
  - **Inbound email processing** — Receiving replies, processing support tickets, parsing bounce notifications, and closing the feedback loop on your email program.

# Why Not Just Run Your Own Mail Server?

- A common question from students with limited infrastructure experience: "Can't I just install an SMTP server on an EC2 instance?" Technically yes, but practically it is a terrible idea for production workloads. Here is why:
  - **IP reputation is earned, not given.** Fresh IP addresses from cloud providers are often pre-flagged by ISPs as suspicious. It takes weeks or months of consistent, legitimate sending to build trust. One mistake can destroy that reputation overnight.
  - **Deliverability is complex.** Major inbox providers (Gmail, Outlook, Yahoo) use machine learning models that evaluate hundreds of signals — your IP history, domain age, content quality, engagement rates, authentication records, and complaint ratios — to decide whether your email reaches the inbox or the spam folder.
  - **Compliance is non-trivial.** CAN-SPAM (US), CASL (Canada), GDPR (EU), and other regulations impose strict rules on commercial email. Managed services help enforce opt-out mechanisms and record-keeping.
  - **Scaling is hard.** Sending 10 emails per minute is straightforward. Sending 10,000 per minute without getting throttled or blacklisted requires careful IP warming, connection pooling, and retry logic.
- Managed email services like AWS SES handle all of this infrastructure for you: IP warming, ISP feedback loops, bounce processing, authentication signing, and compliance tooling.

# How Email Delivery Works — The Lifecycle of a Message

# From "Send" to "Inbox" — What Actually Happens

**Key Vocabulary**

- **MX Record** — A DNS record that specifies which mail server handles email for a domain. When SES sends an email to user@example.com, it looks up the MX record for example.com to find where to deliver it.
- **SMTP (Simple Mail Transfer Protocol)** — The protocol used to transfer email between servers. It dates back to 1982 (RFC 821) and was designed with essentially zero security — no sender verification, no encryption, no authentication. Every security mechanism we discuss today exists because SMTP was built on trust.
- **Envelope vs. Header** — An email has two "from" addresses. The envelope sender (Return-Path/MAIL FROM) is used by servers for routing and bounce handling. The header "From" is what the human recipient sees. These can be different — and this gap is exactly what attackers exploit.

This is where your DNS knowledge from earlier lectures directly applies. SPF records, DKIM keys, DMARC policies, and MX records are all DNS TXT or MX records.

Misconfiguring DNS is the single most common reason email authentication fails.

```
Your Application
    |
    v
Email Service (e.g., AWS SES)
    |  - Authenticates sender (SPF, DKIM)
    |  - Checks suppression list
    |  - Applies sending quotas
    v
Outbound SMTP Server
    |  - DNS lookup for recipient's MX record
    |  - TLS connection to recipient's mail server
    v
Recipient's Mail Server (e.g., Gmail, Outlook)
    |  - SPF check: Is this IP authorized?
    |  - DKIM check: Is the signature valid?
    |  - DMARC check: Does policy pass?
    |  - Spam filtering: Content, reputation, engagement
    v
Inbox  /  Spam Folder  /  Rejected (Bounce)
```

# Email Authentication — SPF, DKIM, and DMARC

# The Problem: SMTP Has No Built-In Identity Verification

- SMTP was designed in an era when everyone on the internet was a trusted academic.
- There is no built-in mechanism to verify that the sender is who they claim to be.
- Anyone can connect to an SMTP server and claim to be ceo@yourcompany.com.
- This is why we need authentication protocols layered on top.

# SPF — Sender Policy Framework

- SPF answers the question: **"Is this IP address allowed to send email on behalf of this domain?"**
- How it works:
  - The domain owner publishes a DNS TXT record listing all IP addresses and services authorized to send email for that domain.
  - When a receiving server gets an email claiming to be from example.com, it looks up the SPF record for example.com.
  - If the sending server's IP is in the authorized list, SPF passes. Otherwise, it fails.
- **Critical limitations of SPF:**
  - SPF only checks the **envelope sender** (Return-Path), not the "From" header that humans see. An attacker can pass SPF with their own domain in the envelope while spoofing your domain in the visible "From" header.
  - SPF has a **10 DNS lookup limit**. Each include: directive triggers a DNS lookup. Organizations using many third-party email services (marketing tools, CRM, transactional email, helpdesk) can easily exceed this limit. When you exceed it, SPF returns a permanent error and effectively stops working — this is one of the most common silent failures in email authentication.
  - SPF **breaks on forwarding**. When an email is forwarded, the forwarding server's IP is not in your SPF record, so the check fails.

# Example SPF Record

This says: "Amazon SES and Google Workspace are authorized to send on our behalf. Reject everything else (-all)."

```
v=spf1 include:amazonses.com include:_spf.google.com -all
```

# DKIM — DomainKeys Identified Mail

- DKIM answers the question: **"Was this email actually sent by the claimed domain, and has it been tampered with in transit?"**
- How it works:
  - The sending server signs the email with a private cryptographic key.
  - The corresponding public key is published as a DNS TXT record.
  - The receiving server retrieves the public key from DNS and verifies the signature.
- This is asymmetric cryptography applied to email — the same concept behind TLS/SSL certificates and SSH keys. The signature covers the email headers and body, so any modification in transit (injecting content, changing headers) invalidates the signature.
- **Key points about DKIM:**
  - DKIM **survives forwarding** (unlike SPF), because the signature is attached to the message itself, not the sending IP.
  - DKIM requires a **selector** — a label that lets you have multiple DKIM keys for different services (e.g., one for SES, one for Google Workspace).
  - If you do not configure a custom DKIM key, your email provider signs emails with their own domain. The signature is valid, but it does not match your "From" domain — which means DKIM alignment fails for DMARC purposes.

# DMARC — Domain-based Message Authentication, Reporting & Conformance

- DMARC answers the question: **"What should receiving servers do when SPF or DKIM checks fail, and who should they report failures to?"**
- DMARC is the **policy layer** that ties SPF and DKIM together. It adds two critical capabilities:
- **Alignment** — DMARC requires that the domain authenticated by SPF or DKIM matches (aligns with) the domain in the visible "From" header. This closes the gap that SPF alone leaves open.
- **Policy enforcement** — The domain owner specifies what to do with failing messages:
  - p=none — Do nothing, just send me reports (monitoring mode)
  - p=quarantine — Put failing messages in the spam folder
  - p=reject — Reject failing messages outright
- **The alignment requirement is crucial.** SPF and DKIM can both individually pass, but if neither authenticated domain aligns with the visible "From" header domain, DMARC still fails. This is the most commonly misunderstood aspect of email authentication.

Example DMARC record:

```
_dmarc.example.com  TXT  "v=DMARC1; p=reject; rua=mailto:dmarc-reports@example.com"
```

# How All Three Work Together

```
Incoming Email
    |
    +---> SPF Check: Is sender IP authorized? -----> Pass/Fail
    |         |
    |         +---> SPF Alignment: Does envelope domain
    |                match "From" header domain?
    |
    +---> DKIM Check: Is signature valid? ----------> Pass/Fail
    |         |
    |         +---> DKIM Alignment: Does signing domain
    |                match "From" header domain?
    |
    +---> DMARC Evaluation:
              Did at least one check (SPF or DKIM) BOTH
              pass AND align?
                  |
                  Yes --> Deliver
                  No  --> Apply policy (none/quarantine/reject)
```

# Email Delivery Issues — Bounces, Complaints & Suppression

# Bounces

- A bounce occurs when an email cannot be delivered to the recipient. There are two types:
- **Hard bounces** are permanent delivery failures. The mailbox does not exist, the domain is invalid, or the recipient server has permanently rejected the message. You should never retry delivery to an address that hard bounces. AWS SES does not retry hard bounces (except for DNS lookup failures) and automatically adds hard-bounced addresses to your account-level suppression list.
- **Soft bounces** are temporary delivery failures. The recipient's mailbox is full, the receiving server is temporarily overloaded, or the connection timed out. SES retries soft bounces multiple times over a period. If delivery still fails after all retries, SES stops attempting.
- Bounces can also be **synchronous** (rejected during the SMTP conversation while servers are still communicating) or **asynchronous** (the recipient server initially accepts the message, then later generates a bounce-back notification).

# Complaints

- A complaint occurs when a recipient marks your email as spam. Most email clients have a "Mark as Spam" or "Report Spam" button. When a recipient clicks it, the ISP records the complaint and, if a feedback loop is established, sends the complaint back to your email service.
- This is the most damaging signal for your sender reputation. A complaint means a human actively told their ISP that they consider your email unwanted. Even a small complaint rate (above 0.1%) can trigger warnings, and rates above 0.3% can cause serious deliverability problems.

# The Suppression List

- AWS SES maintains a global suppression list of email addresses that have recently caused hard bounces for any SES customer. If you try to send to an address on this list, SES treats it as a hard bounce without actually attempting delivery. This protects both your reputation and the broader SES infrastructure.
- Key facts about the suppression list: addresses can remain on it for up to 14 days, bounces from the suppression list still count toward your bounce rate, and you can submit a removal request if you are certain the address is valid.

# AWS SES Account Health Model

- AWS SES actively monitors two critical metrics for every account:
  - **Bounce rate** — If your bounce rate exceeds 5%, SES issues warnings. If it exceeds 10%, SES may pause your sending ability.
  - **Complaint rate** — If your complaint rate exceeds 0.1%, SES issues warnings. If it exceeds 0.5%, SES may pause your sending.
- The enforcement process has two stages. First, your account is placed **under review** — you can still send, but you must fix the underlying issue. If the issue is not resolved during the review period, SES **pauses** your sending ability entirely. In severe cases (e.g., sending to spam traps), SES can skip the review stage and pause immediately.
- This is not optional or negotiable. Because SES customers share IP address pools, one customer's bad behavior can damage deliverability for all SES users on those IPs.

# Sender Reputation — Building and Protecting It

# What Is Sender Reputation?

- Sender reputation is a score that ISPs and email providers assign to your sending IP addresses, domains, and email addresses. It determines whether your emails reach the inbox, the spam folder, or get rejected entirely. Think of it like a credit score for email — it takes a long time to build and can be destroyed very quickly.
- Factors that influence reputation include: bounce rate, complaint rate, spam trap hits, sending volume consistency, engagement metrics (open rates, click rates), authentication pass rates (SPF, DKIM, DMARC), and content quality.

# Best Practices for Maintaining Reputation

- **Use subdomains to segment email types.** Send marketing emails from marketing.example.com and transactional emails from orders.example.com. Each subdomain develops its own reputation. If your marketing campaigns trigger spam complaints, your transactional emails (password resets, order confirmations) are protected because they use a different subdomain.
- **Warm up new IP addresses and domains gradually.** Do not send 100,000 emails on day one from a new IP. Start with small volumes (hundreds per day) and increase gradually over weeks. ISPs are suspicious of sudden high-volume sending from unfamiliar sources.
- **Maintain clean recipient lists.** Remove addresses that hard bounce immediately. Remove addresses that have not engaged (opened or clicked) in the past 6 months. Use double opt-in for new subscriptions — send a confirmation email that requires the recipient to click a link before they are added to your list.
- **Never send to purchased or scraped email lists.** These lists invariably contain spam traps — email addresses operated by anti-spam organizations specifically to catch senders who do not practice proper list hygiene. Hitting a spam trap can result in immediate blacklisting.
- **Monitor your metrics continuously.** Set up CloudWatch alarms on your SES bounce and complaint rates. Do not wait for AWS to notify you — by then, damage may already be done.

# Spam Traps — The Silent Reputation Killer

- Spam traps are email addresses that should never receive legitimate email. They come in several varieties: **pristine traps** are addresses that have never been used by a real person and exist only to catch scraped or purchased lists; **recycled traps** are addresses that once belonged to real users but have been inactive for years and were repurposed as traps by ISPs; **typo traps** exploit common misspellings (e.g., user@gmial.com).
- Sending to a spam trap tells ISPs that your list hygiene is poor, and they will penalize your reputation accordingly. AWS SES monitors for spam trap hits and may place your account under review if they detect them.

# Designing a Cloud-Native Email Architecture

# Putting It All Together

```
                      Your Application
                             |
             +--------------+--------------+
             |                             |
   Transactional Email           Marketing Email
 (orders.example.com)          (marketing.example.com)
             |                             |
             v                             v
         AWS SES                     AWS SES / SendGrid
             |                             |
             +-------- DNS Config ---------+
             |                             |
  SPF: include:amazonses.com    SPF: include:sendgrid.net
  DKIM: Custom key per subdomain
  DMARC: p=reject on example.com (with rua reporting)
             |
             v
     Monitoring & Feedback
             |
  +----+----+----+----+
  |    |    |    |    |
 SNS  SQS  S3  CW  Lambda
  |              |
Bounce/      CloudWatch
Complaint    Alarms:
Processing   - Bounce rate > 5%
             - Complaint rate > 0.1%
```

# Configuration Checklist for Production Email

- **Domain verification** — Verify your sending domain in SES (or your chosen provider)
- **SPF record** — Publish a TXT record authorizing your email service. Keep it under 10 DNS lookups. Use -all (hard fail), not ~all (soft fail)
- **DKIM signing** — Generate and publish DKIM keys for each sending service. Use your own domain, not the provider's default
- **DMARC policy** — Start with p=none and rua reporting. Monitor for 2-4 weeks, then move to p=quarantine, then p=reject
- **Bounce handling** — Configure SNS notifications for bounces. Automatically remove hard-bounced addresses from your sending list
- **Complaint handling** — Configure SNS notifications for complaints. Immediately stop sending to addresses that complain
- **Suppression list** — Use SES account-level suppression to prevent resending to known-bad addresses
- **Monitoring** — CloudWatch alarms on bounce rate, complaint rate, and sending quota usage
- **Subdomain separation** — Use different subdomains for transactional vs. marketing email
- **Move out of sandbox** — Request production access from AWS with a clear explanation of your use case

# Common Mistakes to Avoid

- **Publishing multiple SPF records for the same domain.** SPF requires exactly one TXT record. Having two invalidates both.
- **Using ~all (soft fail) instead of -all (hard fail) in SPF.** Soft fail is treated as "suspicious but deliver anyway" by many ISPs.
- **Forgetting to update DNS when rotating DKIM keys.** Your IT team rotates keys in the email provider but forgets the DNS record — DKIM fails overnight for all outgoing email.
- **Not monitoring DMARC aggregate reports.** You published a DMARC record with rua but never look at the reports. Meanwhile, someone is spoofing your domain.
- **Sending from noreply@ addresses.** Recipients cannot report issues, and replies bounce — which counts against your bounce rate.

# Summary

# Key Takeaways

- Email in cloud-native applications is far more than "just sending messages." It involves a complex interplay of DNS-based authentication protocols (SPF, DKIM, DMARC), sender reputation management, ISP feedback loops, and continuous monitoring. Misconfiguration can range from annoying (emails landing in spam) to catastrophic (millions of phishing emails sent in your company's name).
- The key takeaways are: SMTP has no built-in security, so authentication protocols must be layered on top; SPF authorizes sending IPs, DKIM provides cryptographic message integrity, and DMARC ties them together with policy enforcement; sender reputation is earned slowly and destroyed quickly — monitor bounce and complaint rates continuously; cloud providers handle different parts of the email stack differently — AWS has SES, GCP recommends third-party services; and real-world incidents show that even Fortune 100 companies with enterprise email security products get this wrong when defaults are not secure.

# Additional References

- AWS SES Documentation — Sending best practices: https://docs.aws.amazon.com/ses/latest/dg/tips-and-best-practices.html
- AWS SES — Monitoring sender reputation: https://docs.aws.amazon.com/ses/latest/dg/monitor-sender-reputation.html
- Google Email Sender Guidelines: https://support.google.com/a/answer/81126
- Yahoo Sender Best Practices: https://senders.yahooinc.com/best-practices/
- Microsoft Threat Intelligence — Domain spoofing via misconfigurations (Jan 2026): https://www.microsoft.com/en-us/security/blog/2026/01/06/phishing-actors-exploit-complex-routing-and-misconfigurations-to-spoof-domains/
- Guardio Labs — EchoSpoofing report: https://labs.guard.io/echospoofing-a-massive-phishing-campaign-exploiting-proofpoints-email-protection-to-dispatch-3dd6b5417db6
- DKIM specification: http://dkim.org
- SPF specification: http://www.openspf.org
- DMARC specification: https://dmarc.org

# Additional Resources

See course lecture page on Canvas for supplementary materials and documentation links.

LVX
VERITAS
VIRTVS